AI-Assisted Software Development

Module 2: Getting Started with aider

Mike Burton 17-19 March 2025

Prerequisites

Before installing aider:

- Python 3.8+ installed
- Git installed and configured
- OpenAI API key
- Basic command line familiarity
- Code editor of choice

Installation

Step 1: Install aider using pip:

```
pip install aider-chat
```

Step 2: Set up your OpenAI API key:

```
export OPENAI_API_KEY='your-api-key-here'
```

Tip: Add this to a `.env` file for persistence

Common Installation Issues:

1. Python version mismatch
   - Check version: `python --version`
   - Use pyenv if needed
2. pip not found
   - Install pip if needed
   - Ensure pip is in PATH
3. Permission issues
   - Use `--user` flag
   - Or use virtual environment

Creating Your First aider Environment

You can either install aider:

- globally; or
- in a virtual environment

To install globally:

1. Install pipx. (pipx lets you install Python applications globally while keeping them isolated.)
2. Then install aider-chat:

```
pipx install aider-chat
```

To use a virtual environment:

- Aider will be available within that environment

```
# Create a new directory
mkdir aider-project
cd aider-project

# Create and activate virtual environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate

# Install aider in the virtual environment
pip install aider-chat
```

Basic Configuration

aider looks for configuration in several places:

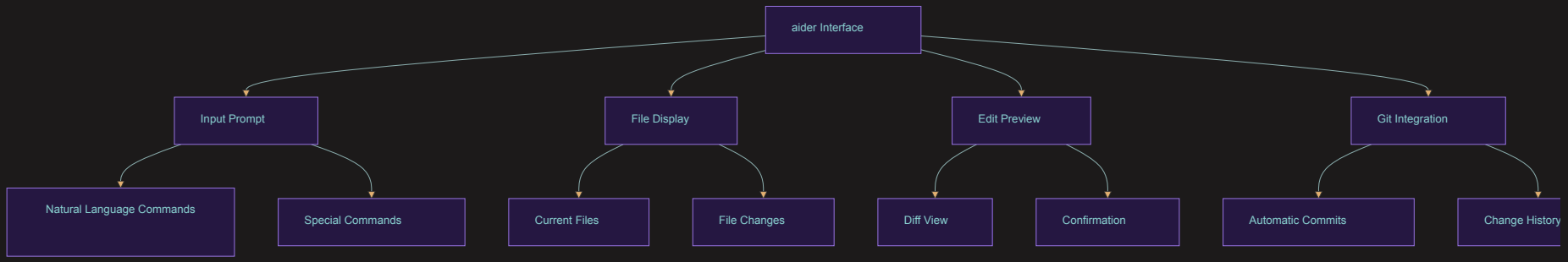
1. Command line arguments
2. Environment variables
3. Configuration file (`~/.config/aider/config.yml`)

Example `config.yml`:

```yaml
model: gpt-4
edit_format: diff
auto_commits: true
git_commit_prefix: "aider: "
```

Understanding the Interface

- aider's interface is based on a simple text prompt
  - you can use natural language to interact with aider
  - you can also use special commands



Starting aider

Basic startup:

```
aider .
```

With specific files:

```
aider path/to/file1.py path/to/file2.py
```

With specific model:

```
aider --model gpt-4 .
```

The prompt will appear:

```
Using git repo: /path/to/your/project
Added 2 files to chat: file1.py, file2.py
Chat with your code using GPT-4
>
```

Basic Commands

Essential aider commands:

Navigation:

- `/help` - Show help
- `/files` - List files
- `/ls` - List tracked files

File Selection:

- `/add <file>` - Add file(s)
- `/drop <file>` - Remove file(s)

Actions:

- `/commit` - Commit changes
- `/undo` - Undo last change
- `/diff` - Show changes
- `/exit` - Exit aider

Your First Interaction

Let's create a simple Python function:

1. Start aider:

```
aider hello.py
```

2. Request a function:

```
> Create a function that greets a user by name
```

Your First Interaction

3. Review the changes:

```
def greet_user(name):
    """
    Greets the user with their name.

    Args:
        name (str): The name of the user to greet

    Returns:
        str: A personalized greeting
    """
    return f"Hello, {name}! Welcome to our application."
```

4. Changes are automatically committed if approved

Working with Multiple Files

Example: Creating a simple web application

1. Start aider with multiple files:

```
aider app.py templates/index.html
```

2. Request functionality:

```
> Create a Flask app with a homepage that displays a greeting
```

3. aider will modify both files coherently

Working with Commits

There are three ways to handle commits:

1. Auto-commit
2. Manual confirmation
3. Manual commits

Auto-commit

Auto-commit is the default behaviour

- Changes are committed automatically

```
aider .
```

Manual confirmation

With manual confirmation, you will be asked to confirm changes before they are committed:

```
aider --no-auto-commit .
...
Apply changes to hello.py? ([y]es, [n]o, [e]dit, [d]iff) [y]:
```

Manual commits

With manual commits, you will edit files directly and then commit them:

- Edit files directly
- Use `/commit` to commit changes
- Optionally add a commit message:

```
/commit Add input validation to user registration
```

Multi-line Messages

To send multi-line messages to aider:

1. Start typing your message
2. Press Enter to add new lines
3. End with a blank line (double Enter)

Example:

```
> I need a Python function that:
- Takes a list of numbers
- Filters out negative numbers
- Returns the sum of remaining numbers
- Includes error handling for non-numeric inputs


```

(blank line above ends the message)

Tip: You can use any text formatting - bullet points, numbered lists, or plain text. aider will understand your request.

Canceling Output

If aider's output isn't what you need:

- Press CTRL-C to cancel the current output
- The conversation will continue normally
- No changes will be made to your files
- You can rephrase your request and try again

Example:

```
> Add input validation
Certainly! I'll add input validation to... ^C
Operation cancelled.
> Let me be more specific. Add email format validation to the user_registration function...
```

Best Practices:

- Don't hesitate to cancel if the response isn't useful
- Rephrase your request to be more specific
- Use CTRL-C early if you see the response going in the wrong direction
- Cancel long outputs if you spot issues early

Understanding Edit Formats

aider supports different edit formats:

1. Diff Format (default):

```
- old code
+ new code
```

2. Whole File Format:

```
File contents will be replaced entirely
```

Set your preference:

```
aider --edit-format diff
# or
aider --edit-format whole
```

Git Integration

aider automatically:

- Tracks changes
- Creates commits
- Maintains history

Example commit message:

```
aider: Added user greeting function with docstring

- Created greet_user function
- Added input validation
- Included documentation
```

Best Practices for Getting Started

1. Start with Small Changes
    - Begin with simple tasks
    - Verify each change
    - Build complexity gradually
2. Use Version Control
    - Always work in a Git repository
    - Review commits regularly
    - Keep changes atomic
3. Maintain Clear Communication
    - Be specific in requests
    - Provide context when needed
    - Ask for clarification
4. Review Changes Carefully
    - Check code before accepting
    - Test functionality
    - Understand modifications

Exercise: Setting Up a Project

Let's practice setting up a new project:

1. Create a new directory
2. Initialize a virtual environment
3. Install aider
4. Create a new Python file
5. Make your first code request
6. Review and commit changes

Recap: Getting Started with aider

- Installation and setup process
- Basic configuration options
- Essential commands
- Working with files
- Git integration
- Best practices
- Troubleshooting

Next Steps

- Learn advanced commands
- Explore complex workflows
- Understand pattern matching
- Work with larger codebases

Questions to consider:

- How will you organize your projects?
- What types of tasks will you start with?
- How will you integrate aider into your workflow?